

Validación cruzada

Ciencia de datos para la toma de decisiones I

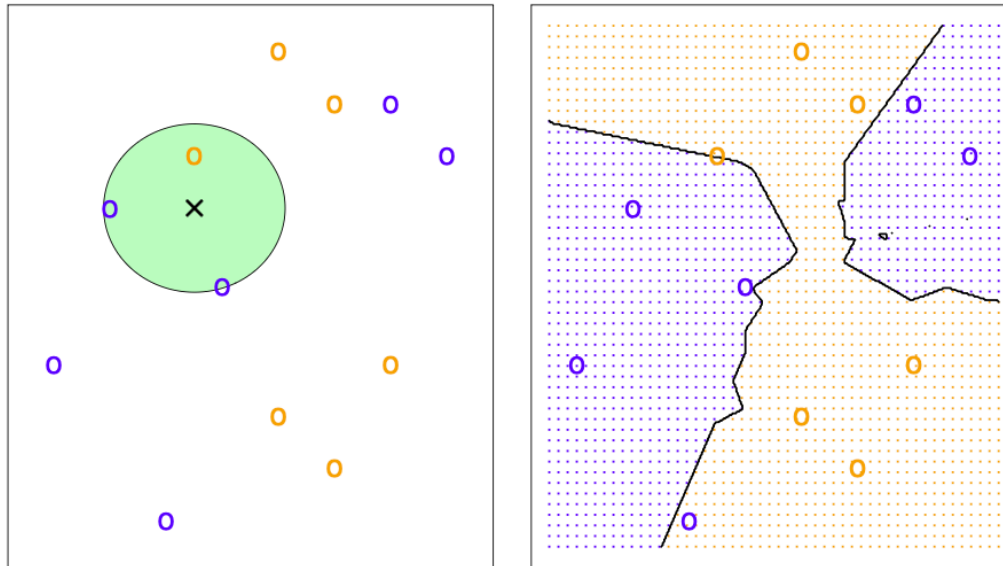
Jorge Juvenal
Campos
Ferreira

✉ juvenal.campos@tec.mx

K-vecinos cercanos

K-vecinos cercanos

El método de k-vecinos cercanos es un modelo de **clasificación supervisada**, basado en **distancia**, que permite clasificar observaciones nuevas basado en la proximidad que tenga un punto nuevo a clasificar con sus k vecinos más cercanos.



¿Que es K-NN?

Algoritmo de aprendizaje supervisado propuesto formalmente por Fix y Hodges (1951) y extendido por Cover y Hart (1967).

Doble proposito

- * **Clasificación:** asigna una categoria (p.ej. fraude / no fraude).
- * **Regresión:** predice un valor continuo (p.ej. precio de casa).

Lazy learner

No construye un modelo explicito durante el entrenamiento. Memoriza los datos y solo calcula al momento de predecir.

No parametrico

No asume ninguna forma de distribución de los datos. Esto le permite capturar fronteras de decisión muy flexibles, sin imponer supuestos como linealidad o normalidad.

Idea clave: K-NN predice mirando a los vecinos mas parecidos del nuevo punto en el conjunto de entrenamiento.

La intuición

"Dime con quien andas y te diré quien eres."

Premisa

Si un nuevo punto se parece a sus vecinos en el espacio de características, probablemente pertenezca a su misma clase.

Idea central

Proximidad implica similitud.

Toda la mecánica del algoritmo se reduce a medir distancias y consultar a los vecinos.

El espacio de características

$$\mathbf{x} = (x_1, x_2, \dots, x_n)$$

Cada observación es un vector con n atributos

Geometria

Cada observación es un punto en un espacio de n dimensiones. Con 2 variables: plano. Con 3: cubo. Con n : hipercubo.

Cercanía = métrica

"Cercanía" no es algo abstracto: se mide con una fórmula de distancia que asigna un número a cada par de puntos.

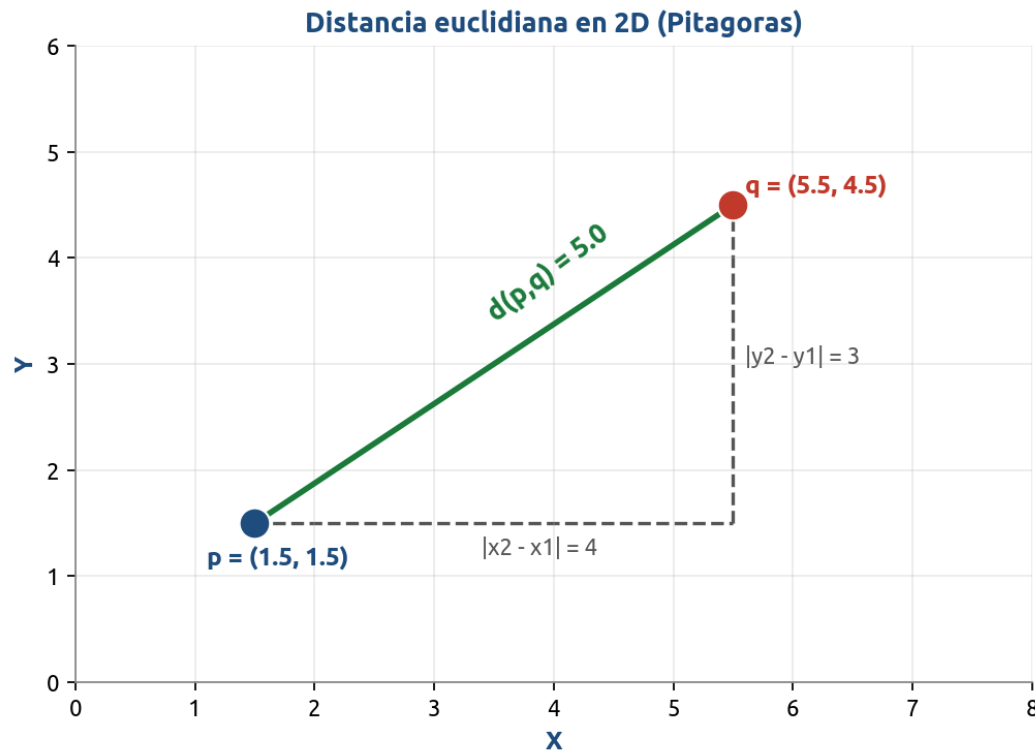
Ejemplo (3 variables)

Un municipio podría describirse con (PIB per capita, índice de pobreza, escolaridad promedio). Cada uno es un punto en R^3 y municipios parecidos quedarían cerca entre sí.

Distancia euclidiana (la mas usada)

$$d(p,q) = \sqrt{\sum_{i=1..n} (p_i - q_i)^2}$$

Generalizacion de Pitagoras a n dimensiones



Notacion

- * p y q : dos puntos en $\mathbb{R}^n$
- * p_i, q_i : i -esima coordenada
- * n : numero de variables

Caso 2D (Pitagoras)

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Otras métricas de distancia

Metrica	Formula	Cuando usarla
Manhattan (L1)	$\sum p_i - q_i $	Variables tipo grilla, robusta a outliers
Minkowski	$(\sum p_i - q_i ^p)^{1/p}$	Generaliza L1 y L2 (parametro p)
Coseno	$1 - (p \cdot q) / (\ p\ \ q\)$	Texto, datos de alta dimensionalidad

Como elegir la metrica?

No hay una respuesta unica: depende del tipo de variables y del dominio. La euclidiana es el punto de partida por defecto; Manhattan ayuda con outliers; coseno funciona bien con texto.

El algoritmo, paso a paso

Paso 0: definir k (entero positivo, idealmente impar en clasificación binaria).

1

a) Calcular

Distancia entre el nuevo punto x y cada observación del conjunto de entrenamiento.

2

b) Ordenar

Ordenar las distancias de menor a mayor.

3

c) Seleccionar

Tomar los k vecinos mas cercanos a x .

4

d) Predecir

Clasificación: clase mayoritaria entre los k vecinos.
Regresión: promedio del valor objetivo.

Todo el costo computacional vive en la predicción: por eso K-NN es rápido de entrenar pero lento en predicción sobre datasets grandes.

Voto mayoritario (formalización)

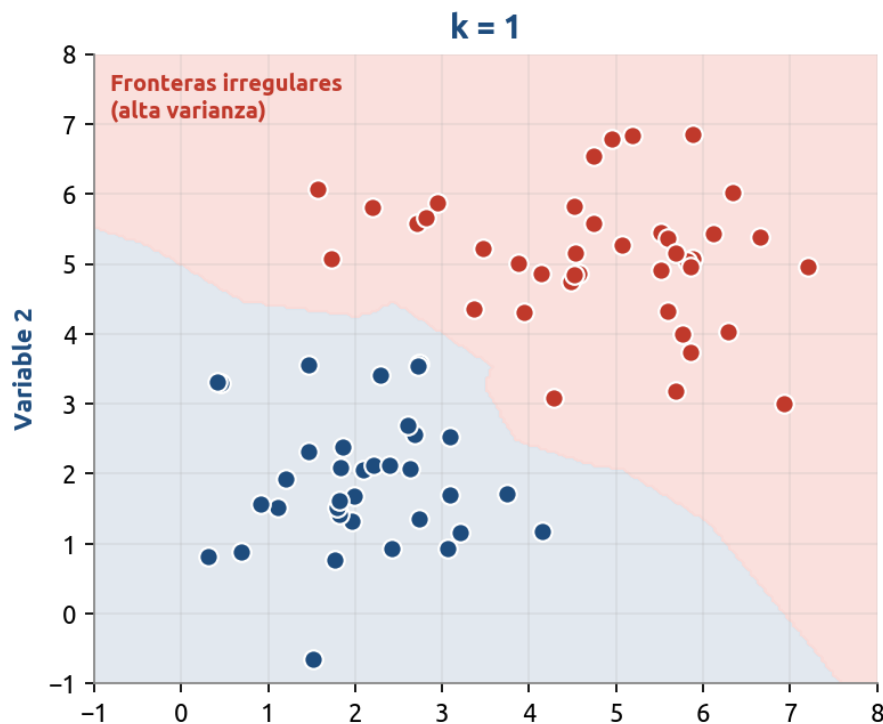
Para clasificación, la predicción para un punto x es:

$$y(x) = \arg \max_{c \in \mathcal{C}} \sum_{i \in N_k(x)} 1(y_i = c)$$

Términos:

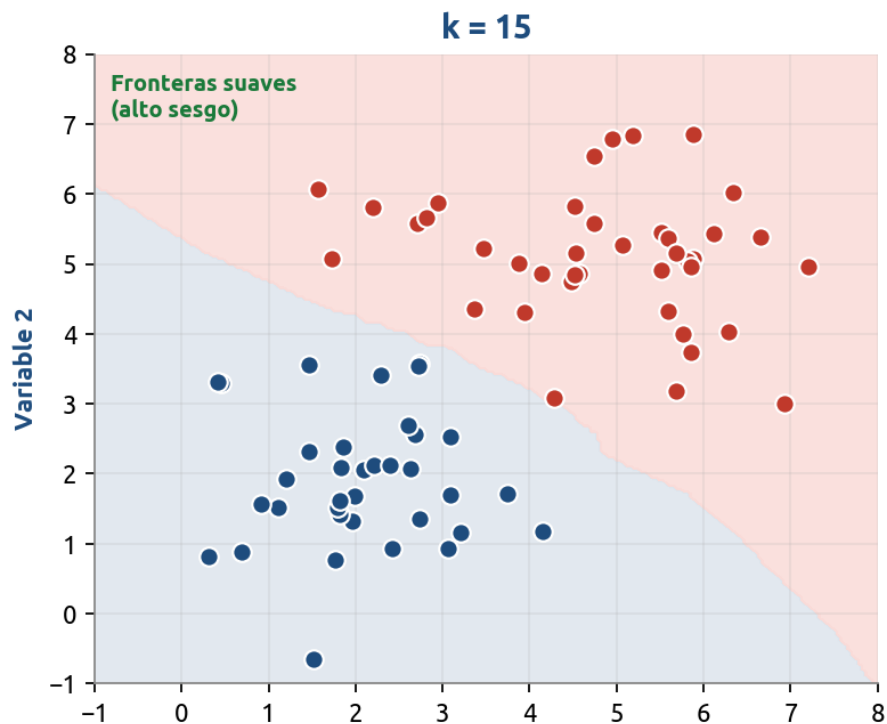
- x : el punto que queremos clasificar.
- $y(x)$: la clase predicha para x .
- $\mathcal{C}$: el conjunto de clases posibles.
- $N_k(x)$: los k vecinos más cercanos a x según una métrica de distancia.
- y_i : la clase verdadera del vecino i .
- $1(y_i = c)$: función indicadora — vale 1 si el vecino i es de clase c , 0 si no.
- $\sum 1(y_i = c)$: cuenta cuántos vecinos son de clase c .
- $\arg \max_c$: devuelve la clase con la cuenta más alta.

Como elegir k?



k pequeno (p.ej. k = 1)

Fronteras irregulares, alta varianza, sensible al ruido -> sobreajuste.

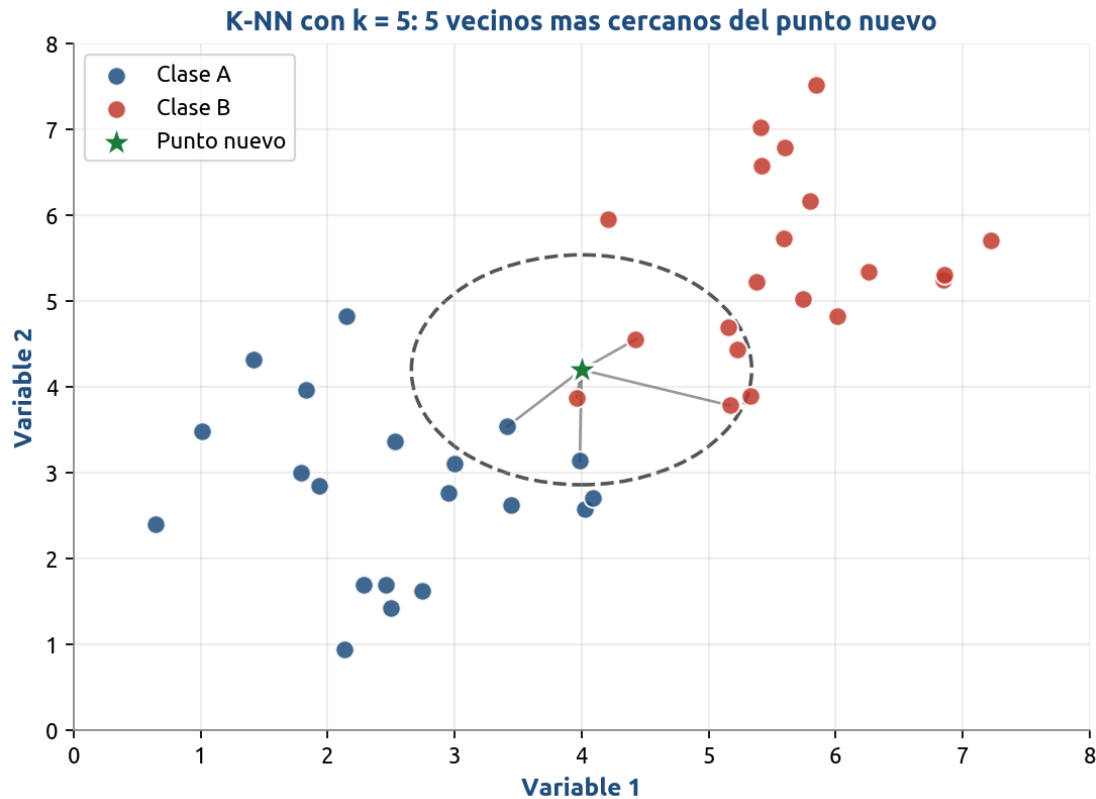


k grande

Fronteras suaves, alto sesgo -> riesgo de subajuste.

Reglas practicas: empezar con $k \sim \sqrt{n}$, elegir k impar en binaria, y elegir el k final con validación cruzada.

Visualizacion de K-NN



Lectura

El punto verde (nuevo) consulta a sus 5 vecinos mas cercanos. El circulo punteado marca el radio.

Decision

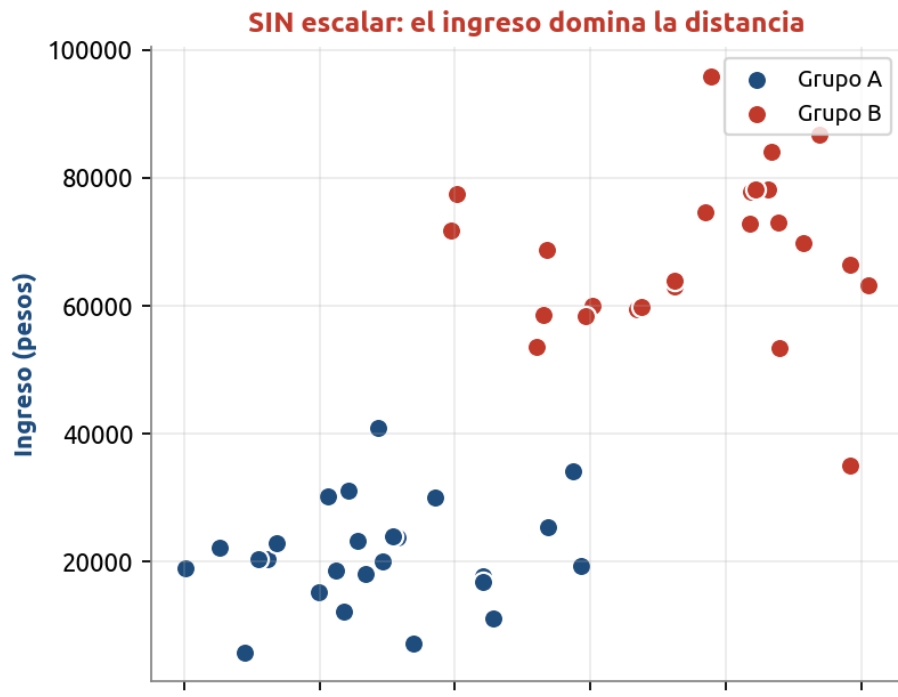
Si la mayoria de esos 5 vecinos pertenece a la Clase B, el punto se clasifica como Clase B.

Importante

Cambiar k cambia el radio del circulo y, con el, la predicción.

Escalar las variables es OBLIGATORIO

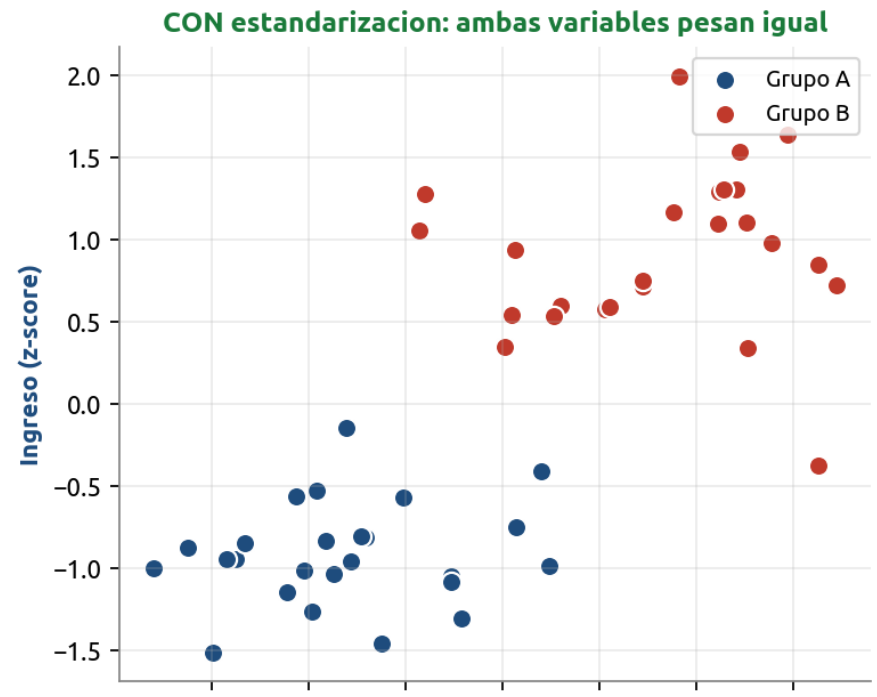
Las variables con escalas grandes dominan el calculo de distancia.



Estandarizacion (z-score)

$$x' = (x - \mu) / \sigma$$

Media 0 y desviacion 1.



Normalizacion min-max

$$x' = (x - x_{\min}) / (x_{\max} - x_{\min})$$

Todo queda en [0, 1].

Ventajas y desventajas de K-NN

Conceptualmente simple

Fáciles de entender e interpretar: 'mira a tus vecinos y decide'. Útil para comunicar a audiencias no técnicas.

Sin fase de entrenamiento

No hay función de costo que optimizar ni parámetros que ajustar; basta con almacenar los datos.

Multi-clase natural

Maneja sin esfuerzo problemas con tres o más categorías: el voto mayoritario se generaliza directamente.

No paramétrico

No asume ninguna distribución subyacente; se adapta a la estructura real de los datos.

Fronteras no lineales

Captura fronteras de decisión arbitrariamente complejas sin tener que especificarlas a priori.

Versátil

Sirve tanto para clasificación como para regresión, con la misma lógica subyacente.

Costoso en predicción

Complejidad $O(n \times d)$ por punto a predecir: se vuelve lento con muchos datos o muchas variables.

Maldición de la dimensionalidad

En alta dimensión todas las distancias se vuelven similares y el concepto de 'vecino cercano' pierde sentido.

Requiere mucha memoria

Hay que guardar todo el conjunto de entrenamiento; no hay un modelo compacto que reemplace los datos.

Sensible a escalas

Una variable mal escalada puede dominar el cálculo y arruinar las predicciones.

Variables irrelevantes danan

K-NN no distingue por sí solo variables informativas de ruido: todas suman a la distancia.

Sensible al desbalance

Clases con muchos ejemplos tienden a 'ganar' los votos, sesgando la predicción contra las minoritarias.

Variantes y mejoras

Weighted K-NN

Cada voto se pondera por el inverso de la distancia ($w_i = 1/d_i$). Los vecinos mas cercanos influyen mas que los lejanos, reduciendo el impacto del ruido.

KD-Trees / Ball-Trees

Estructuras de datos que permiten buscar a los k vecinos en tiempo sub-lineal en lugar de comparar contra todos los puntos. Muy efectivas en dimensionalidad moderada.

Approximate Nearest Neighbors (ANN)

Algoritmos aproximados que sacrifican exactitud por velocidad: FAISS, Annoy, HNSW. Permiten K-NN sobre millones o miles de millones de puntos.

Reducción de variables previa

Aplicar PCA, selección por feature importance u otras técnicas para reducir la dimensionalidad ANTES de K-NN, mitigando la maldición de la dimensionalidad.

Casi todas las mejoras atacan dos problemas: velocidad de búsqueda o calidad de la métrica en alta dimensión.

Workflows

Workflows - Juntando todo

Supongamos que queremos probar 2 modelos distintos; una regresión logística y un k-vecinos cercanos (knn). En ese caso, añadiríamos un paso adicional:

Paso 1: Declaramos los modelos:

```
# Regresión logística (la que ya tenías)
```

```
modelo_logistico <- logistic_reg() %>%  
  set_engine("glm") %>%  
  set_mode("classification")
```

```
# KNN — neighbors lo dejamos tuneable
```

```
modelo_knn <- nearest_neighbor(  
  neighbors      = tune(),          # hiperparámetro a buscar  
  weight_func    = "rectangular",  
  dist_power     = 2                # distancia euclidiana  
) %>%  
  set_engine("kknn") %>%  
  set_mode("classification")
```

Workflows - Juntando todo

Paso 2. Generamos un objeto *workflow()*

Workflow con el modelo logístico

```
wf_logistico <- workflow() %>%  
  add_recipe(receta_reservas) %>%  
  add_model(modelo_logistico)
```

Workflow con el knn

```
wf_knn <- workflow() %>%  
  add_recipe(receta_reservas) %>%  
  add_model(modelo_knn)
```

Workflows - Juntando todo

Un **workflow** es un objeto contenedor de tidymodels que empaqueta en una sola pieza los componentes de un pipeline de modelado: el preprocesador (una recipe o una fórmula) y el modelo (una especificación de parsnip).

Mecánicamente, el objeto tiene tres "slots":

```
wf <- workflow() %>%  
  add_recipe(receta_reservas) %>% # slot "pre" -> preprocesamiento  
  add_model(modelo_knn)           # slot "fit" -> modelo
```

Validación cruzada

Workflows - Juntando todo

Mientras el workflow está "vacío" (no se ha llamado `fit()`), guarda **especificaciones**: la receta sin preparar y el modelo sin ajustar.

Cuando llamas `fit(wf, data = training)`, *tidymodels* hace dos cosas en cadena y las guarda dentro del mismo objeto:

- * (1) ejecuta `prep()` sobre la receta usando esos datos, aprendiendo medianas, sd, umbrales de correlación, niveles de los factores, etc., y
- * (2) ejecuta `bake()` para producir la matriz de diseño y ajusta el modelo sobre ella.

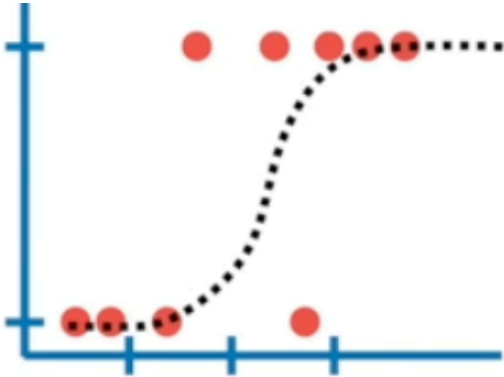
El resultado es un workflow "fitted" o ajustado, que contiene **la receta preparada y el modelo entrenado**, listo para `predict()`.

EL WORKFLOW ES LA UNIDAD MÍNIMA DE TRABAJO EN TIDYMODELS, y es la forma en que trabajaremos de ahora en adelante.

Validación cruzada

Supongamos que queremos hacer un modelo de clasificación:

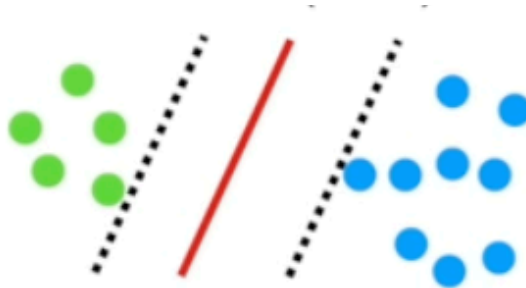
Podríamos usar una regresión logística



O k-vecinos Cercanos



O el algoritmo de Support Vector Machines

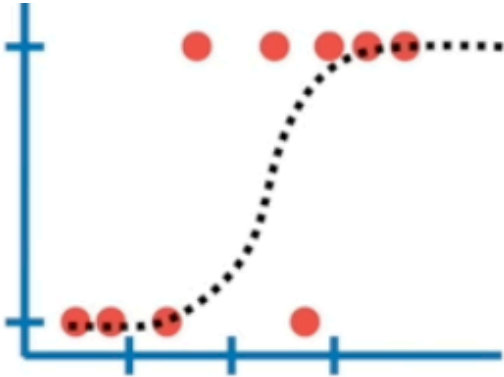


La **validación cruzada** nos permite comparar diferentes métodos de machine Learning para entender como se comportarían en la práctica.

Validación cruzada

Supongamos que queremos hacer un modelo de clasificación:

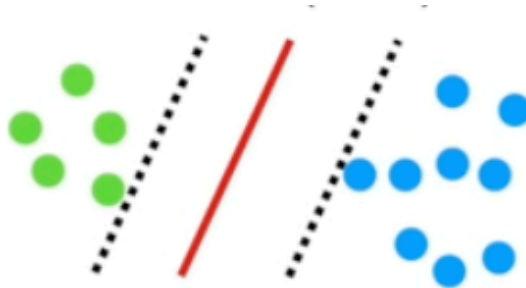
Podríamos usar una regresión logística



O k-vecinos
Cercanos



O el algoritmo de Support
Vector Machines



La **validación cruzada** nos permite comparar diferentes métodos de machine Learning para entender como se comportarían en la práctica.

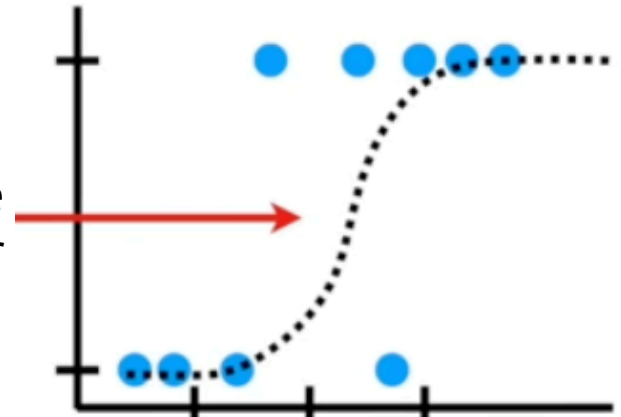
Validación cruzada



Necesitamos hacer dos cosas con los datos:

- 1) **Estimar** los parámetros del método de machine learning
- 2) **Evaluar** que tan bien funciona el modelo de Machine Learning

A la evaluación de un modelo de Machine Learning, también se le puede decir “probar el algoritmo”.



Validación cruzada



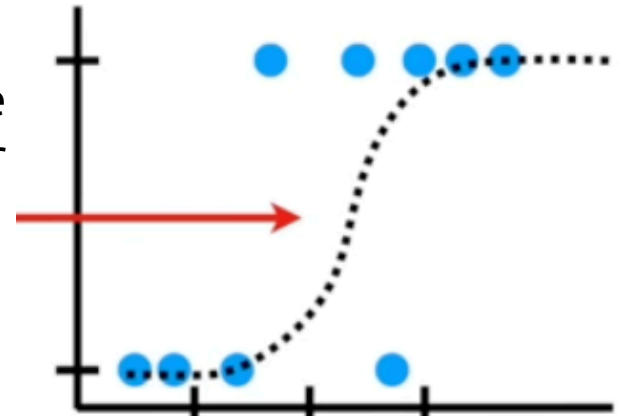
Necesitamos hacer dos cosas con los datos:

- 1) **Estimar** los parámetros del método de machine learning
- 2) **Evaluar** que tan bien funciona el modelo de Machine Learning

A la evaluación de un modelo de Machine Learning, también se le puede decir “probar el algoritmo”.

En resumen, en ML necesitamos:

- 1) **Entrenar** el algoritmo
- 2) **Evaluar o probar** el algoritmo

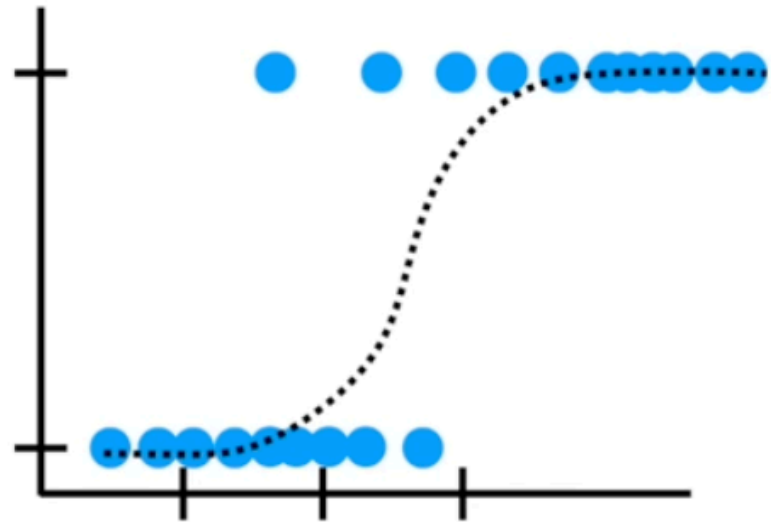


Validación cruzada

No podemos usar los mismos datos para entrenamiento y prueba porque necesitamos saber como se va a comportar el método con datos distintos a aquellos con los que se entrenó.



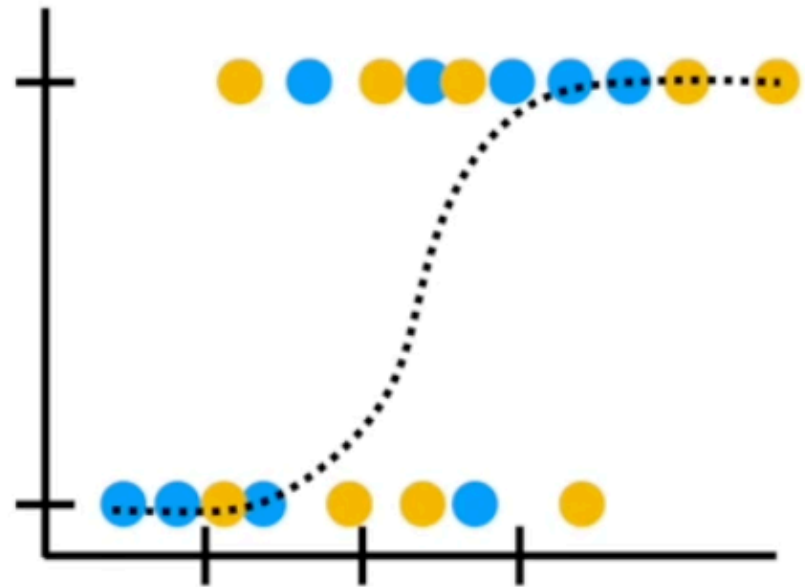
Es como darle las respuestas del examen para que se lo memorice.



Validación cruzada

El enfoque tradicional al iniciar en ML entrenar el modelo con el 75% de los datos...

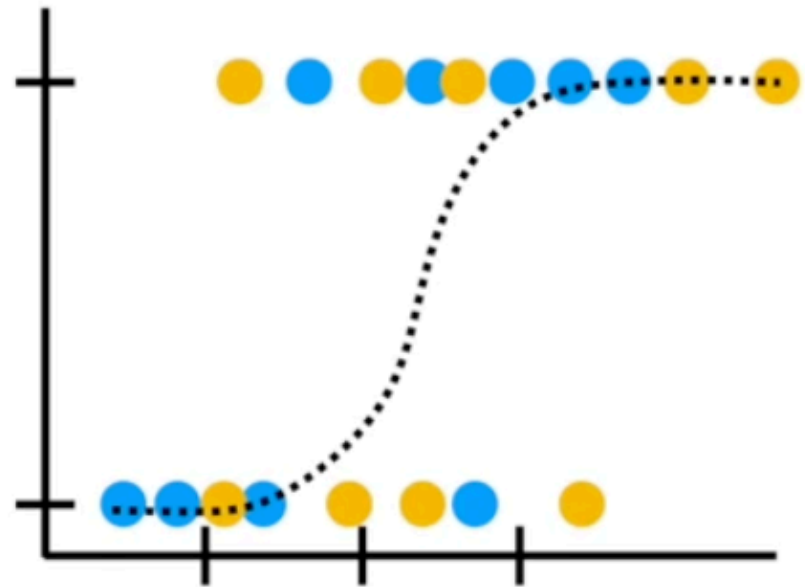
...Y usar el 25% restante para la evaluación del modelo...



Validación cruzada

El enfoque tradicional al iniciar en ML entrenar el modelo con el 75% de los datos...

...Y usar el 25% restante para la evaluación del modelo...



Validación cruzada



Pero ¿cómo sabemos que usar el 75% de los datos para el entrenamiento y el 25% restante es la mejor manera de dividir el dataset?

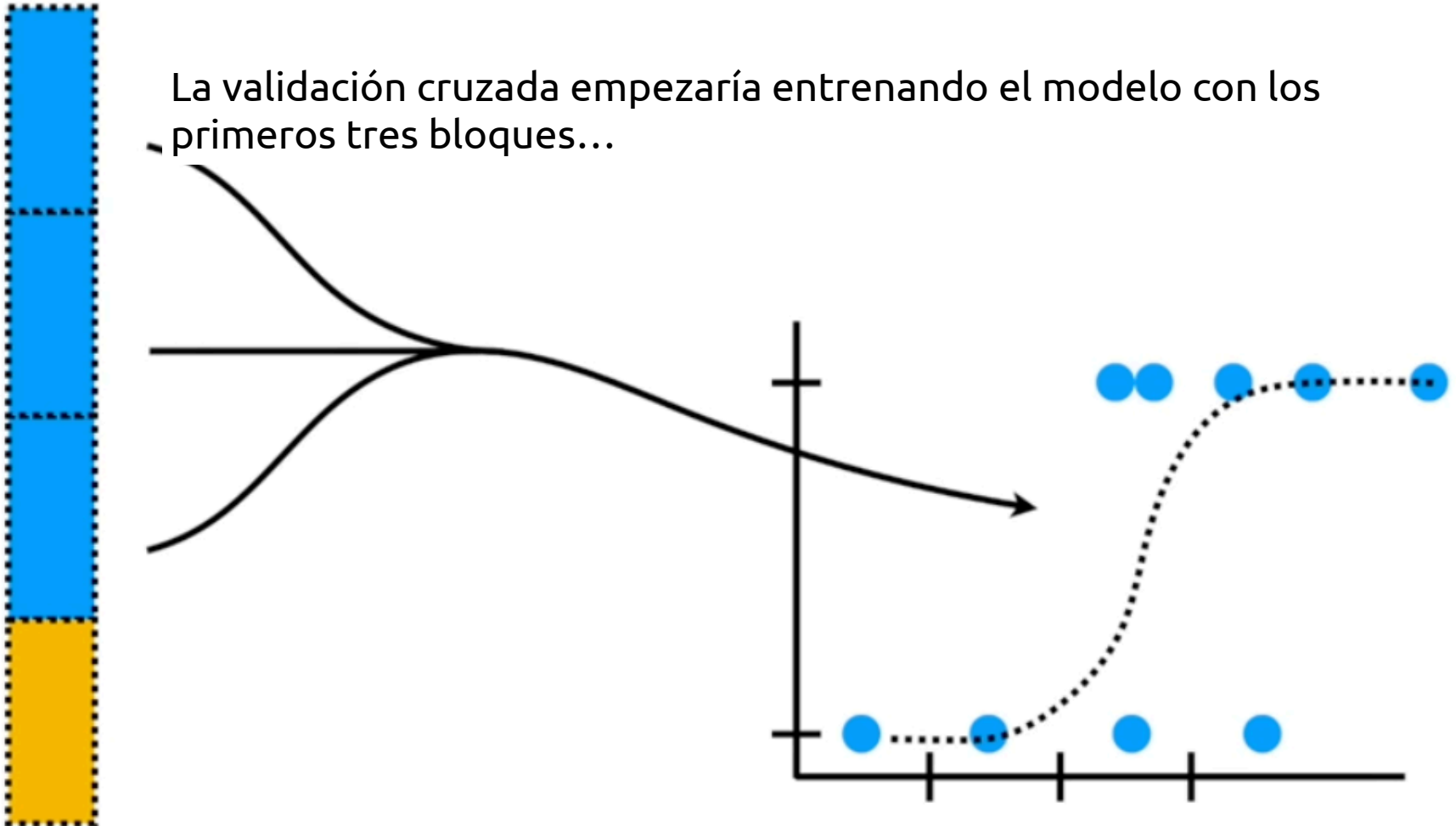
En vez de preocuparnos sobre qué bloque de datos usar cuando dividimos los datos en cachos de 25%, la validación cruzada los usa todos, uno a la vez, y resume los resultados al final.



Validación cruzada

Supongamos que dividimos los datos en cuatro cachos iguales. **Entrenamos** el modelo con los primeros tres cachos (75%) y **evaluamos** el modelo con el cachito restante (25%)

La validación cruzada empezaría entrenando el modelo con los primeros tres bloques...

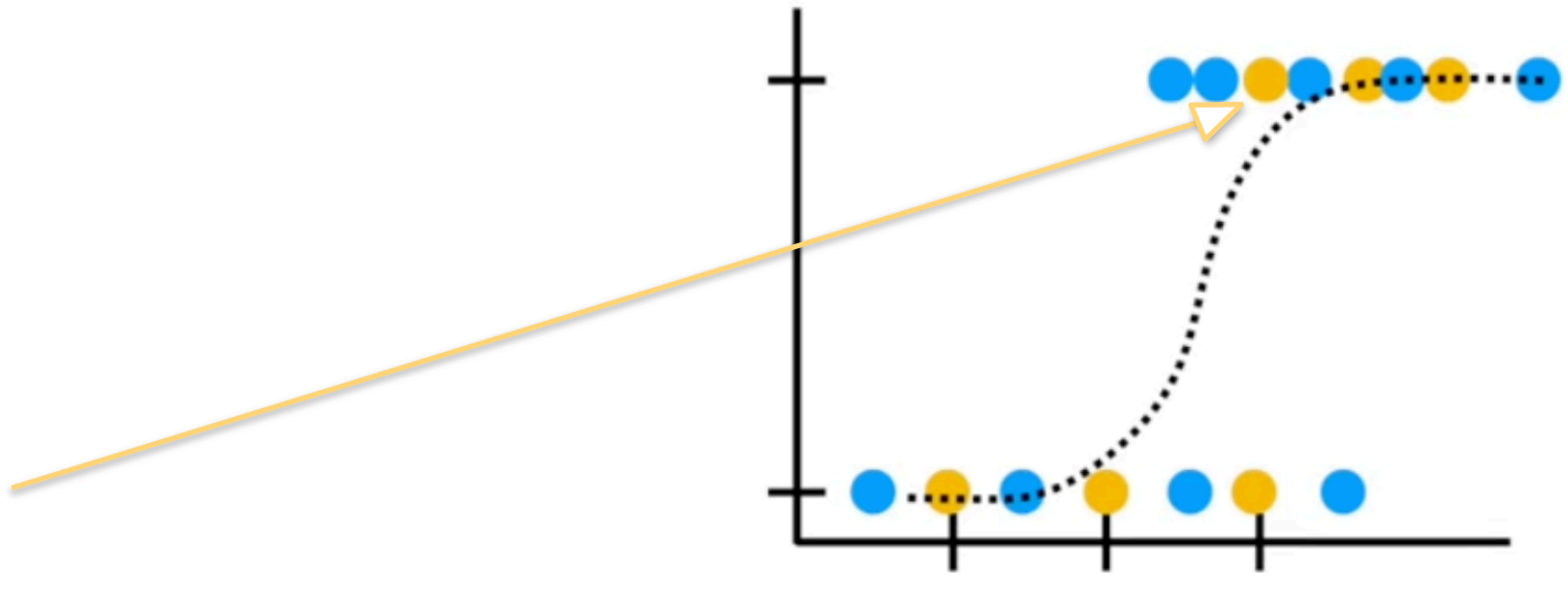


Validación cruzada

Y utilizaría el último bloque de 25% para la evaluación.

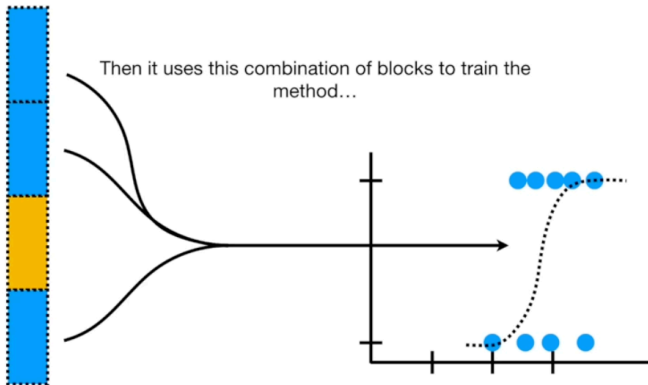
Y mantendría el registro de los resultados de la prueba correspondiente

Test data categorization...	
Correct	Incorrect
5	1



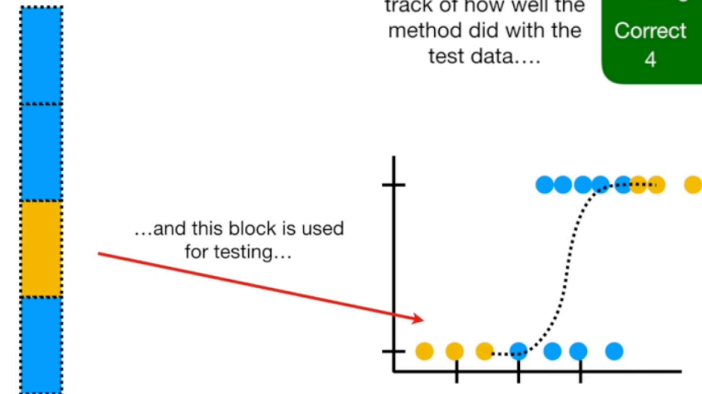
Validación cruzada

Lo mismo para el **bloque 2**

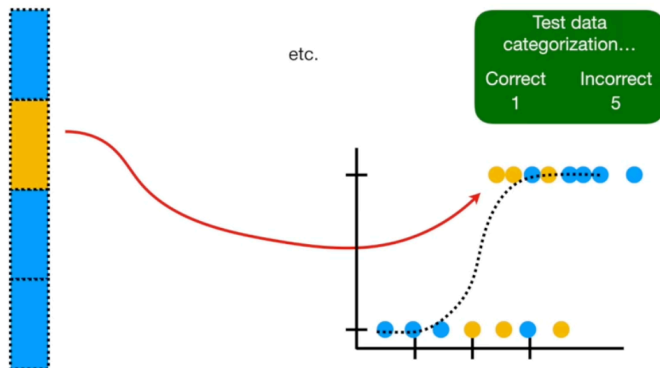


...and then it keeps track of how well the method did with the test data...

Test data categorization...	
Correct	Incorrect
4	2

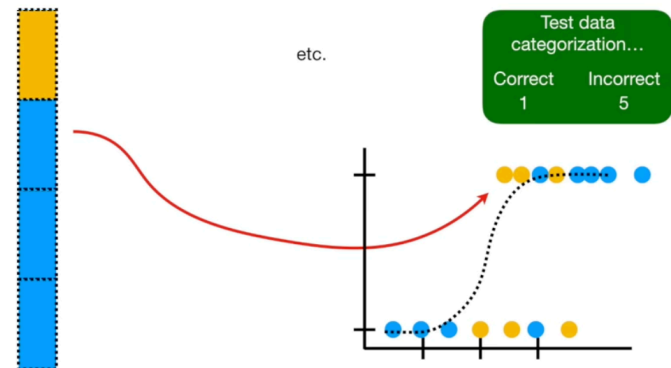


Y para el **bloque 3**



Test data categorization...	
Correct	Incorrect
1	5

Y para el **bloque 4**

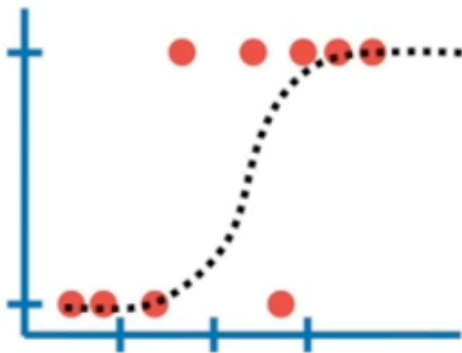


Test data categorization...	
Correct	Incorrect
1	5

Validación cruzada

Al final, cada bloque de datos es usado para en sus respectivas pruebas y así podemos comparar los métodos viendo que también se desempeñaron en estas:

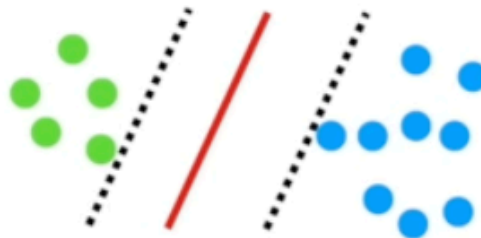
Logistic Regression



Correct
16

Incorrect
8

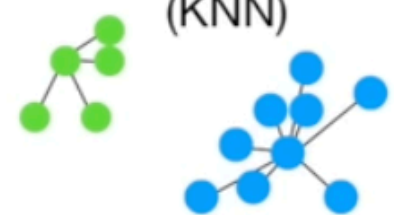
Support Vector machines (SVM)



Correct
18

Incorrect
6

K-nearest neighbors (KNN)



Correct
10

Incorrect
12

Validación cruzada

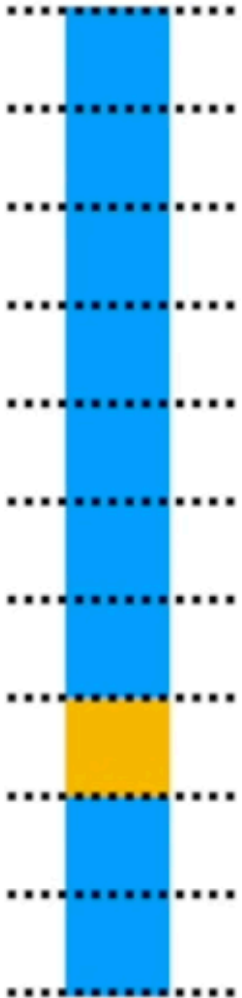
Nota: En este ejemplo, dividimos los datos en 4 bloques.

A esto se le llama “**Validación cruzada de 4 pliegues (Four-Fold Cross Validation)**”.

Sin embargo, el número de bloques o pliegues puede ser cualquiera



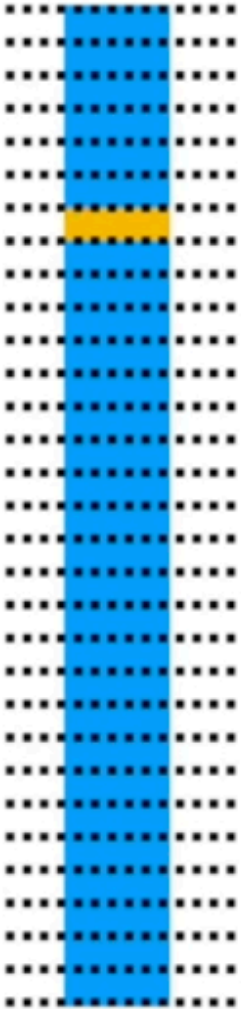
Validación cruzada



En la práctica, es también muy común dividir los datos en 10 bloques.

A esto se le llama **Validación cruzada de 10 pliegues** (Ten-Fold Cross Validation)

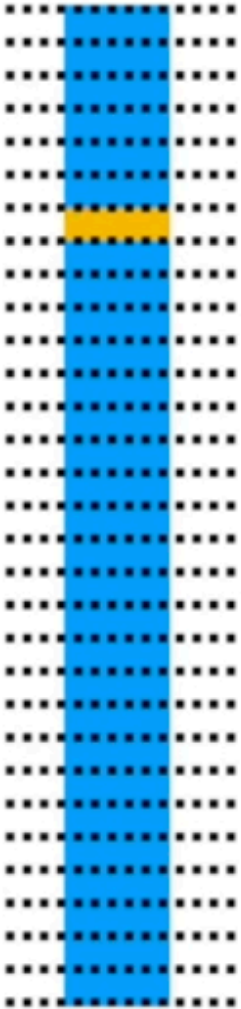
Validación cruzada



El caso más extremo es que cada individuo sea un bloque individual.

A esto se le llama “**Leave One Out Cross Validation (LOOCV)**” y es el método más demandante computacionalmente.

Validación cruzada



La validación cruzada nos sirve para:

1. Afinar las estimaciones de los errores del modelo en la fase de evaluación.
2. Tunear los hiperparámetros
3. Mejorar las comparaciones entre modelos
4. Tunear los hiperparámetros para conseguir un mejor modelo
5. Obtener los valores del umbral de decisión que mejoran el desempeño del modelo

Hiperparámetros

Los **hiperparámetros** son las configuraciones de un modelo de Machine Learning que el algoritmo no aprende a partir de los datos, sino que deben ser definidos de antemano por el analista antes de la construcción del modelo.

Los hiperparámetros se ajustan con un proceso de “Ajuste de hiperparámetros”, que consiste en buscar y seleccionar los valores adecuados para mejorar su rendimiento.

Las regresiones básicas **NO** tienen hiperparámetros. Otros modelos sí (como k-vecinos cercanos).

Gracias